

MATH 231 – Linear Algebra I

Setup Guide 2 ■ Python, pip, and an IDE

Queens College, CUNY ■ Fall 2026 ■ Instructor: Kevin Bedoya

What this guide does. By the end you will have Python installed, the package installer `pip` working, a code editor set up, the `numpy` package installed, and a short program that runs and prints two vectors. That is the complete toolkit for every piece of programming in this course.

No experience assumed. If you have never written a line of code, you are in the right place — Python is genuinely one of the friendlier languages to start with, and programming will never be assessed on an exam in this course. Follow the steps in order and check the **Checkpoint** after each. Section §8 is troubleshooting.

Everything here is free. No licences, no trials, no credit card.

1. What you are installing, and why

Three separate pieces. They are often confused, so it is worth naming them.

Piece	What it is
Python	The language itself, plus the program that runs your code. Without it, a <code>.py</code> file is just text.
pip	Python’s package installer. It fetches libraries other people have written — like <code>numpy</code> — and puts them where Python can find them. It comes bundled with Python.
An IDE	Where you write the code. Technically optional, but it gives you syntax colouring, error highlighting, and a Run button.

Install them in that order: Python first, since `pip` arrives with it, and the IDE last, since it needs to find the Python you installed.

2. Install Python

Find your operating system.

Windows

1. Go to <https://www.python.org/downloads/> and click the big **Download Python 3.x** button.
2. Run the installer. On the very first screen, **tick the box at the bottom that says “Add python.exe to PATH”**.
3. Then click **Install Now** and let it finish.

Watch out. That checkbox is the single most important step in this guide. It is off by default, it is easy to miss, and skipping it is the cause of roughly every “Python is not recognised” message. If you are not sure whether you ticked it, run the installer again and choose *Modify* — reinstalling does no harm.

macOS

Go to <https://www.python.org/downloads/> and run the macOS installer, accepting the defaults. Alternatively, if you already use Homebrew:

```
brew install python
```

Watch out. macOS ships with an old Python that the system uses for its own purposes. Leave it alone. On a Mac, always type `python3` and `pip3` rather than `python` and `pip`.

Linux

Python 3 is usually already installed. Check first, and install only if it is missing:

```
sudo apt update && sudo apt install python3 python3-pip # Ubuntu, Debian, Mint
sudo dnf install python3 python3-pip # Fedora
```

Check that it worked

Open a **new** terminal window — new, because an already-open one will not know about a program installed a minute ago — and run:

```
python --version # Windows
python3 --version # macOS and Linux
```

Checkpoint. You see something like Python 3.12.4. Any version starting with 3. is fine. If it says 2.7, see §8.4.

3. Check pip

There is nothing to install — pip ships with every modern Python. You only need to confirm it is there:

```
python -m pip --version # Windows
python3 -m pip --version # macOS and Linux
```

The `-m` means *run the module named pip using this Python*. It is slightly longer to type than plain `pip`, and worth the habit: it guarantees that packages get installed into the same Python that runs your code. Most beginner package problems come from those two being different.

Checkpoint. You see a line like `pip 24.0 from ... (python 3.12)`.

Optionally, bring pip up to date:

```
python -m pip install --upgrade pip
```

4. Install an IDE

An **IDE** (integrated development environment) is a text editor built for code. Pick *one* of the two below — both are free, both are widely used in industry, and either is completely fine for this course.

Option	Choose it if
VS Code	You want something light and general-purpose that also handles other languages, or you already use it.
PyCharm	You want everything preconfigured for Python specifically, with less setup and more built in.

4.1. VS Code

1. Download from <https://code.visualstudio.com> and install it.
2. Open it and click the **Extensions** icon in the left sidebar (four small squares).
3. Search for **Python** and install the one published by **Microsoft**. This is what makes VS Code understand Python rather than treating your file as plain text.
4. Press **Ctrl+Shift+P** (**Cmd+Shift+P** on a Mac), type **Python: Select Interpreter**, and choose the Python 3 you installed in §2.

4.2. PyCharm

1. Download from <https://www.jetbrains.com/pycharm/download/>.
2. Choose the **Community Edition** — it is free and has everything this course needs. You do not need the paid Professional edition.
3. Install and open it. When you create a new project, PyCharm will find your Python installation on its own.

Checkpoint. You can open your editor, create a file called `test.py`, and see the code coloured rather than uniformly black.

5. Install your first package: `numpy`

A **package** is code somebody else wrote that you can use in your own programs. `numpy` — “Numerical Python” — is the standard package for numerical work, and it is the one we will lean on most this semester, since it is built around exactly the object this course is about: the vector.

Install it:

```
python -m pip install numpy      # Windows
python3 -m pip install numpy     # macOS and Linux
```

It will print a few lines of download progress and finish with a line beginning **Successfully installed numpy**.

Checkpoint. Run the line below. It should print a version number and no errors.

```
python -c "import numpy; print(numpy.__version__)"
```

Later packages. We will add others as the course needs them — `scipy` and `matplotlib` are the likely next two — and each is installed the same way, by swapping the name in the command above. There is nothing to install ahead of time; I will say when.

6. Your first program

Time to check that all three pieces work together.

1. In your editor, create a new file named `main.py`.
2. Type the following into it. Three lines of work, plus the import that makes `numpy` available.

```
import numpy as np
u = np.array([1, 2, 3])
v = np.array([4, 5, 6])
print(u, v)
```

Line by line:

- `import numpy as np` loads the package and gives it the short nickname `np`. Essentially all Python code in the world writes it this way, so it is worth copying the convention.
- `np.array([1, 2, 3])` builds a numpy array — which for our purposes is a vector — from an ordinary list of numbers. We store one in `u` and another in `v`.
- `print(u, v)` writes both to the terminal.

Save the file, then run it. Either press the **Run** button in your IDE, or from a terminal in the same folder as the file:

```
python main.py          # Windows
python3 main.py         # macOS and Linux
```

Checkpoint. The terminal prints exactly this:

```
[1 2 3] [4 5 6]
```

Note that numpy separates components with spaces rather than commas, which is how you can tell a numpy array from an ordinary Python list at a glance.

If you see that output, your setup is complete: Python runs, pip installed a package, the package imported, and your editor can execute a file. Nothing further is needed until the first assignment.

7. Where your code goes

Assignments are submitted through the private GitHub repository set up in **Setup Guide 1**. Save your `.py` files into the matching folder — `homework/`, `projects/`, or `extra-credit/` — inside your `QC_MATH231` folder, then stage, commit, and push:

```
git add .
git commit -m "Finish homework 1"
git push
```

Work that has not been pushed is not visible to me and cannot be graded. If any of that is unfamiliar, it is covered in Setup Guide 1, §7.

8. Troubleshooting

8.1. “python is not recognized” / command not found

```
'python' is not recognized as an internal or external command
```

Almost always the *Add python.exe to PATH* checkbox from §2. Two fixes, in order of effort:

1. **Open a new terminal.** An open terminal does not pick up a program installed after it started.
2. **Try the launcher.** On Windows, `py` often works even when `python` does not:

```
py --version
```

If that works, use `py` in place of `python` everywhere, including `py -m pip install numpy`.

3. **Reinstall.** Run the installer again, choose **Modify**, and make sure the PATH option is ticked.

8.2. Typing python opens the Microsoft Store

A Windows default that intercepts the command when no real Python is on the PATH. Either install Python properly from <https://python.org> with the PATH box ticked, or turn the stub off: **Settings** → **Apps** → **Advanced app settings** → **App execution aliases**, then switch off both `python.exe` and `python3.exe`.

8.3. “pip is not recognized”

Use the longer form, which works whenever Python itself works:

```
python -m pip install numpy
```

8.4. python --version says 2.7

You are on a Mac or an older Linux system where `python` means the legacy version. Use `python3` and `pip3` instead — everywhere, for the whole semester. Python 2 will not run this course’s code.

8.5. “ModuleNotFoundError: No module named numpy”

```
ModuleNotFoundError: No module named 'numpy'
```

numpy went into a different Python than the one running your file. This is the most common cause of confusion when more than one Python is installed. Fix it by installing through the same interpreter that runs your code:

```
python -m pip install numpy
```

If it still fails inside your IDE but works in the terminal, the IDE is pointed at the wrong interpreter. In VS Code, press **Ctrl+Shift+P** → **Python: Select Interpreter** and choose the same Python 3 you have been using. In PyCharm: **Settings** → **Project** → **Python Interpreter**.

8.6. Permission denied while installing

Do not reach for `sudo`. Install into your own user account instead:

```
python -m pip install --user numpy
```

8.7. “SyntaxError” on a line that looks fine

Check the line *above* the one reported — an unclosed bracket or quote makes Python complain about the next line down. Also check that you have not mixed tabs and spaces for indentation; most editors have a setting to insert spaces for tabs.

8.8. The program runs but prints nothing

You probably saved the file without the `print` line, or ran a different file than the one you edited. Confirm the filename in the terminal output matches the file you are looking at, and that you saved before running.

8.9. Still stuck

Email me at Kevin.Bedoya32@stu-mail.qc.cuny.edu or ask on the class Discord. Include:

- your operating system,
- the exact command you ran, and
- the full error message, copied as text or as a screenshot.

Installation problems are ordinary and everyone hits at least one. Sort yours out early — well before the first assignment is due — so that when the deadline arrives the only thing you are thinking about is the linear algebra.